

Sprawozdanie z eksperymentów – optymalizacja parametrów solvera

Opis problemu

Testowany solver to algorytm ewolucyjny pakujący klocki Tetris na planszę 15×15 z penaltą **bad_diagonal** (wysokie kary na przekątnych). Celem jest maksymalizacja sumarycznego score'u (im więcej niekaranych pól zajętych przez klocki, tym lepiej). Każde uruchomienie kończy się po wyczerpaniu limitu iteracji lub braku poprawy przez patience kolejnych kroków.

Punkt startowy – parametry domyślne

Na początku uruchomiono solver z domyślnymi wartościami parametrów:

Parametr	Wartość
starting_states	10
duplications	100
mutations	40
mut_alter	10
mut_remove	100
mut_add	100
mut_shift	50
mut_rotate	50
mut_clear	100

Średni wynik z 10 uruchomień: **136**. Plansza końcowa z jednego z uruchomień:

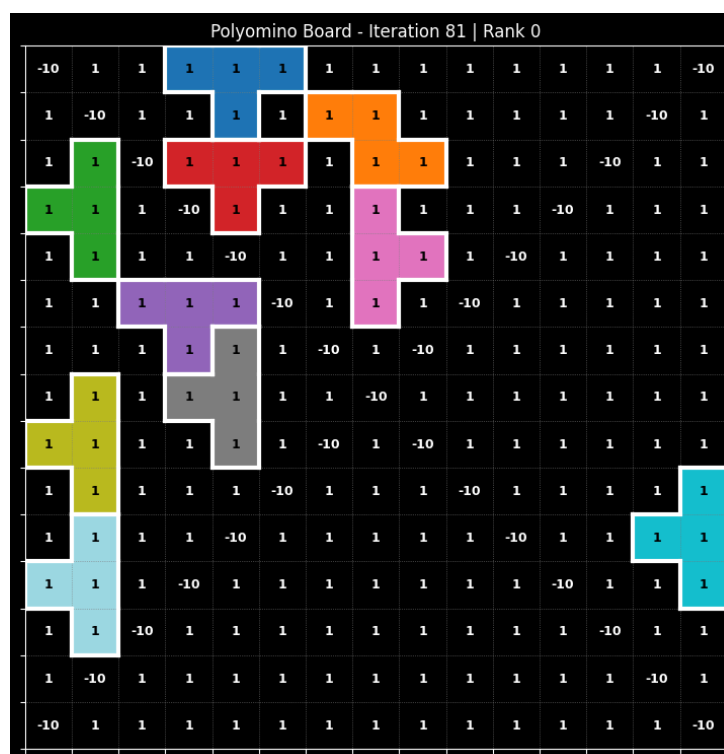


Figure 1: Plansza wynikowa – parametry domyślne (score 134–136). Widać że klocki gęsto pokrywają środek, ale przekątne karzące zostają zajęte.

Wynik nie jest zadowalający – algorytm nie nauczył się skutecznie omijać przekątnych, a eksploracja przestrzeni jest nieefektywna przy tak dużej liczbie duplikacji i mutacji.

Pierwsza runda Grid Search

Przeprowadzono przeszukiwanie siatki parametrów (grid search) z 10 uruchomieniami na kombinację. Najlepsza znaleziona konfiguracja:

Parametr	Wartość
starting_states	20
duplications	10
mutations	10
mut_alter	50
mut_remove	50
mut_add	100
mut_shift	50
mut_rotate	100
mut_clear	50

Średni wynik wzrósł do **207** – poprawa o 52% względem punktu startowego. Wnioski:

- Mniejsza liczba duplikacji (10 zamiast 100) i mniej mutacji na iterację pozwala na dokładniejsze przeszukiwanie.
- Większy udział mut_rotate i mut_alter pomaga klockom „prześlizgiwać się” z przekątnych.

Plansza wynikowa po pierwszym grid search:

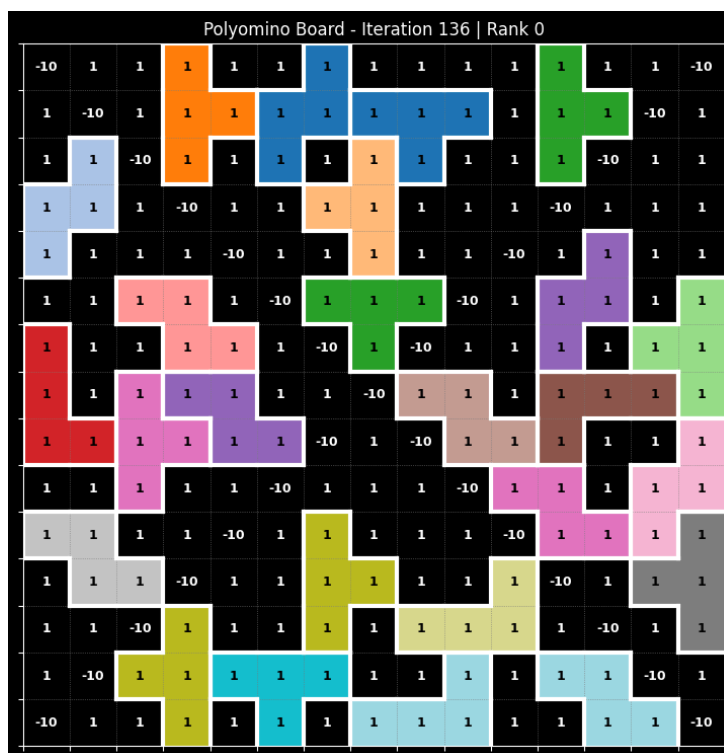


Figure 2: Plansza wynikowa – po 1. grid search (score 194–207). Klocki zaczynają omijać środkowe przekątne.

Kolejne rundy – skupienie na mut_add, mut_rotate i patience

W kolejnych iteracjach grid searcha zawężono przestrzeń poszukiwań, koncentrując się na wagach mutacji dodawania i obrotu klocków oraz na wartości parametru patience. Znaleziona konfiguracja końcowa:

Parametr	Wartość
patience	80
starting_states	20
duplications	10
mutations	2
mut_alter	50
mut_remove	50
mut_add	150
mut_shift	50
mut_rotate	300
mut_clear	50

Średni wynik: **254** – łącznie poprawa o 87% względem domyślnych parametrów. Kluczowe zmiany:

- mut_rotate = 300 – bardzo wysoka waga obrotu sprawia, że algorytm agresywnie reorientuje klocki zamiast ich usuwać.
- mut_add = 150 – częstsze dokładanie nowych klocków.
- mutations = 2 – zaledwie 2 mutacje na dziecko; algorytm robi małe, precyzyjne kroki zamiast chaotycznych skoków.

Plansza wynikowa z najlepszej konfiguracji:

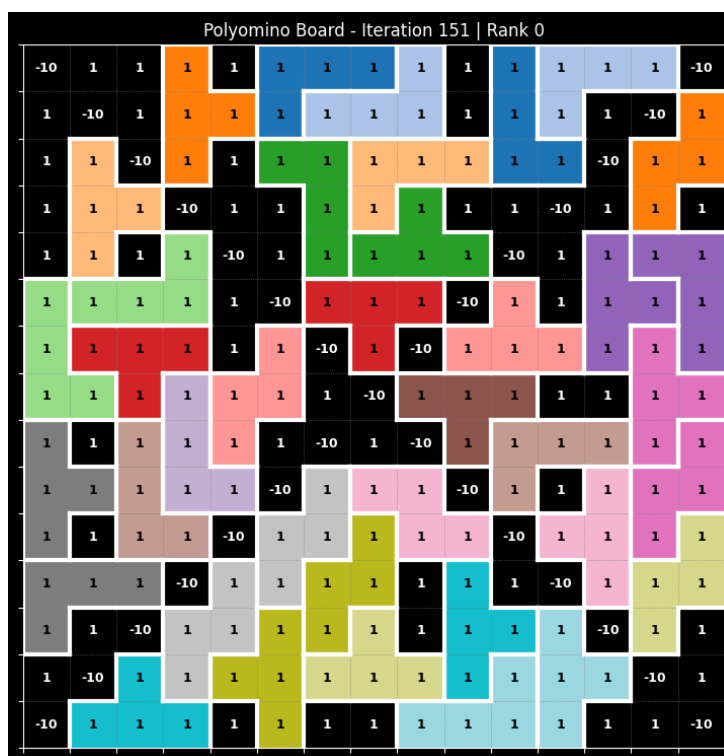


Figure 3: Plansza wynikowa – najlepsze parametry (score 246–254). Klocki wyraźnie omijają przekątne, pokrycie jest gęste w strefach niskiej kary.

Wpływ parametru patience na jakość rozwiązania

Aby sprawdzić, czy patience naprawdę ma istotny wpływ, przeprowadzono dodatkowy eksperyment: dla obu konfiguracji (domyślnej i najlepszej) mierzono średni score przy 12 różnych wartościach patience (5–200), z 10 powtórzeniami na punkt.

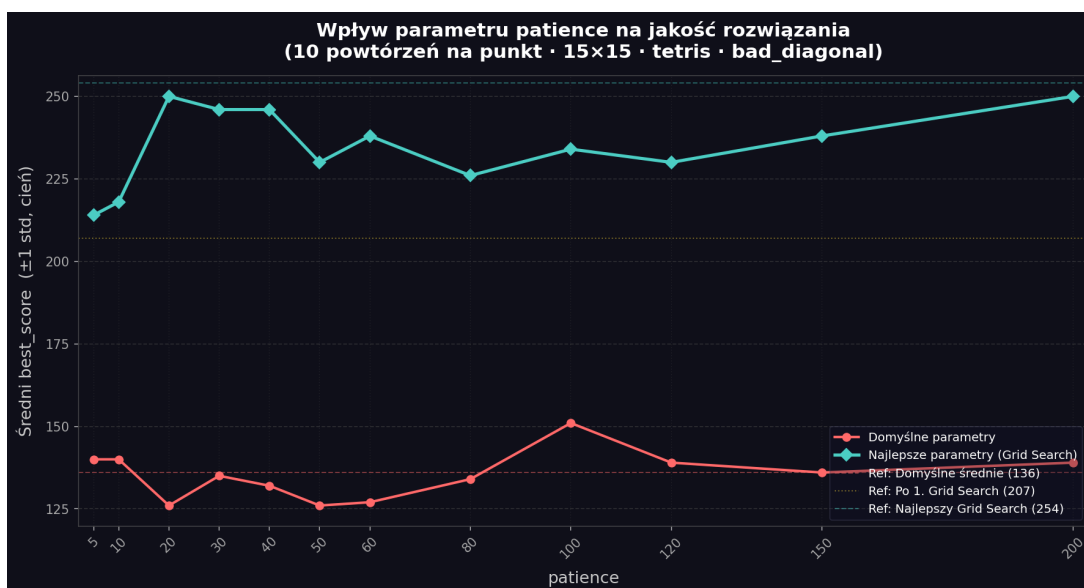


Figure 4: Średni score w funkcji parametru patience. Cień = ± 1 odchylenie standardowe. Poziome linie przerywane to wartości referencyjne z grid searcha.

Obserwacje:

- Dla **domyślnych parametrów** patience nie zmienia wyników w sposób istotny – oscylują one w przedziale 112–146 niezależnie od wartości.
- Dla **najlepszych parametrów** widać wyraźny wzrost wraz z patience do 100, po czym wyniki stabilizują się. Wartość patience = 80–100 jest rozsądnym kompromisem między jakością a czasem działania.
- Powyżej patience = 100 nie widać dalszej poprawy, co sugeruje że algorytm z tymi parametrami osiąga swój naturalny pułap przed wyczerpaniem limitu iteracji.

Podsumowanie

Konfiguracja	Śr. score	Poprawa
Domyślna	136	–
Po 1. grid search	207	+52%
Końcowa (najlepsza)	254	+87%

Kluczową zmianą okazało się drastyczne zwiększenie wagi `mut_rotate` oraz zmniejszenie liczby mutacji na dziecko (`mutations = 2`), co zmieniło charakter algorytmu z chaotycznego na precyzyjny. Parametr `patience = 80` okazał się wystarczający – wyższe wartości nie przynoszą wymiernych korzyści.